

利用 GPU 进行十亿规模的相似性搜索

Jeff Johnson¹, Matthijs Douze, and H. J.égou, Senior Member, IEEE

摘要—相似性搜索适用于处理图像或视频等复杂数据的数据库系统，这些数据通常由高维特征表示，需要特定的索引结构。本文探讨了如何在这项任务中更好地利用 GPU 的问题。虽然 GPU 擅长距离计算等数据并行任务，但在这一领域的先前方法却因暴露较少并行性的算法（如 h-min 选择）或对内存层次结构的利用不佳而陷入瓶颈。我们提出了一种新颖的 *h*-选择设计。通过优化基于乘积量化的暴力搜索、近似搜索和压缩域搜索，我们将其应用于不同的相似性搜索场景。在所有这些设置中，我们的性能都大大优于现有技术。我们的实现以高达 55% 的理论峰值性能运行，使近邻实现比现有 GPU 技术快 8.5 $\times$ 。在 35 分钟内就能在 YFCC100M 数据集的 9500 万张图像上构建高精度 h-NN 图，在 4 个 Maxwell Titan X GPU 上不到 12 小时就能构建连接 10 亿个向量的图。为了便于比较和复制，我们已将我们的方法开源。

索引术语—相似性搜索、多媒体数据库、索引方法、图形处理单元

C

1 引言

IMAGES 和视频构成了索引和搜索所需的新的海量数据源。传统的媒体管理系统基于结构化数据建立的关系数据库。例如，图像是根据拍摄时间和地点等元数据以及可能的人工添加（如图像中的人名）编制索引的。因此，可以根据姓名、日期或地点对图像进行查询。这些元数据可以自动整理相册。

对于大型媒体收藏来说，这种元数据更难以获得；制作内容是如此容易，以至于数据注释成为一个重大瓶颈。目前正在使用各种机器学习和深度学习算法来自动解释和注释这些复杂的现实世界实体。它们产生的表示或嵌入，通常是 50 到 1000 多个维度的实值高维向量。流行的例子包括文本表示 word2vec [43] 和 FastText [34]、从卷积神经网络中提取的图像表示 [25], [49] 以及用于实例搜索的图像描述符 [7], [26], [53]。传统的关系数据库无法有效处理这些描述符，因为它们需要模糊匹配、分类器和相似性搜索等机器学习工具。

许多向量表示只能在 GPU 系统上有效生成，因为底层进程要么具有很高的算术复杂性和/或高数据带宽需求[36]，要么由于通信开销或表示问题而无法有效分区。

质量[48]。一旦制作完成，对它们的操作本身就是计算密集型的。如何利用 GPU 资产并不简单。更广泛地说，如何利用新的异构磁盘/CPU/GPU/FPGA 架构是数据库界的一个关键课题[11]。

在这种情况下，通过数字相似性而非结构关系进行搜索更为合适。这可能是为了找到与图片最相似的内容，也可能是为了找到对线性分类器响应最高的向量。在大型数据集上执行的最昂贵的操作之一是计算 h-NN 图，这是一个有向图，数据库中的每个向量都是一个节点，每条边都连接着一个节点和它的 *h* 个近邻。这是我们的主要应用。最先进的方法（如 NN-Descent[18]）在完成这项任务时，会在数据集本身的基础上产生大量内存开销，因此无法轻松扩展到我们所考虑的十亿级数据库。

此类应用必须应对维度诅咒[57]，这使得穷举搜索和非穷举搜索的精确索引在十亿规模的数据库中都不切实际。这就是为什么有大量工作涉及近似搜索和/或图构建。为了处理 RAM 中无法容纳的庞大数据集，有几种方法使用编码对向量进行压缩表示。这对于像 GPU 这样内存有限的设备来说尤其方便。事实证明，接受最小的精度损失就能带来数量级的压缩[28]。最流行的矢量压缩方法可分为二进制编码 [24], [29] 或量化方法 [32], [47]。这两种方法都有一个理想的特性，即搜索相邻数据时不需要重建向量。

传统的关系数据库存储在磁盘上。不过，我们的目标是，在访问数百兆字节数据的操作中，响应时间能达到 10 毫秒左右。因此，我们只考虑存储在 RAM 中的数据库。

¹J. Johnson 是 Facebook AI Research 的一员，地址：New York, NY 10003 USA。

电子邮件：jhh@fb.com。

M. Douze and H. J.égou are with Facebook AI Research, Paris, France. 电子邮件：{matthijs, rvou@matthijs, rvj}@fb.com。

收稿日期：2017 年 10 月 30 日；修订日期：2019 年 4 月 18 日；接受日期：2019 年 5 月 23 日。发表日期：2019 年 6 月 7 日；当前版本日期：2021 年 6 月 29 日。（通讯作者：

Jeff Johnson）。

推荐 接受 Y. Xia. Digital Object Identifier no. 10.1109/TBDATA.2019.2921572

2332-7790/© 2019 IEEE. 允许个人使用，但再版/再发行须经 IEEE 许可。
更多信息，参见 <https://www.ieee.org/publications/rights/index.html>。

现代服务器通常拥有数十亿到数百亿字节的内存，可以处理数十亿条目的数据集，如果分布在多台服务器上，则可以处理数万亿条目的数据集。因此，在下文中，我们关注的是给定内存预算下的搜索速度。磁盘 I/O 性能与此无关。

本文重点讨论基于乘积量化 (PQ) 代码的方法。与位置敏感散列 (LSH) [15] 或其他产生二进制编码的变体[24]、[37] 相比，这些方法被证明更为有效。LSH 的二进制编码变体是次优的，因为它丢弃了矢量分量的规范，而 PQ 则不同，它满足 Lloyd 的量化器最优条件。LSH 也被用作一种分割技术。其中，E²LSH 提供了一套适用于欧几里得搜索的哈希函数[2]。不过，这种方法需要将整个数据集编码成多个表，内存开销很大。相比之下，PQ 变体对每个向量只使用一个代码。LSH 和 PQ 的并排比较验证了 PQ 的优越性 [27]、[45]。当查询向量没有编码时，PQ 尤其有效。该算法有一个自然扩展，可以进行非穷举搜索 [32]。

与最初的基于 PQ 的技术相比，人们提出了几种改进方案，但大多数都难以在 GPU 上高效实现。倒置多指数[5]适用于高速/低质量操作点，但它依赖于复杂的 "多序列" 算法。优化乘积量化或 OPQ [23] 是对输入向量的线性变换，可提高产品量化的精度；可作为预处理应用。Andr'e 等人[3]的 SIMD 优化实现只能在次优参数（几个粗量化中心点）下运行。其他许多方法，如 LOPQ 和 Polysemous 代码[20]、[35]，都过于复杂，无法在 GPU 上高效实现。

在 GPU 上实现相似性搜索的方法有很多，但大多采用二进制代码 [46]、小型数据集[55]或穷举搜索[17]、[50]、[51]。据我们所知，只有 Wieschollek 等人的研究成果[58]似乎适合使用量化代码的十亿规模数据集。这是在 GPU 上的现有技术水平，我们将在第 6.4 节中在最大（十亿规模）的公共相似性搜索基准上与之进行比较。

本文的贡献如下：

- GPU *h* 选择算法，在快速注册中运行。
我们提供了复杂度分析。迄今为止，这一直是类似 GPU 数据库应用的限制因素；
- 在多 CPU 线程上执行时，近乎最优的算法布局可实现精确和灵活的计算。
在 GPU 上进行近似 *h* 近邻搜索；
- 一系列实验表明，这些改进在以下方面大大优于之前的技术
中型到大型近邻搜索任务。
单 GPU 或多 GPU 配置。

本文算法的精心设计实现可以在开源的 Faiss 库中找到。它实现了许多最先进的索引方法，而且最相关的算法都被翻译成了 GPU 算法。Faiss 库 (<https://github.com/facebookresearch/>)

faiss) 包含重现本文大部分结果的源脚本。

本文介绍的设计在 Faiss 中实现后，已被广泛应用于各种任务。在最近的一项自然语言处理工作[38]中，*h*-*NN* 搜索被用于匹配词嵌入，在没有平行文本的情况下提供不同语言之间的翻译。Caron 等人[14]将 GPU 聚类方法用于嵌入式的非超视化训练。在 [19] 和 [21] 中，巨大的 *h*-*NN* 图被用于图像分类。这些应用之所以能够实现，要归功于 GPU 实现所带来的数量级的性能提升。

本文的结构如下。第 2 节介绍了背景和符号。第 3 节回顾了 GPU 架构，并讨论了使用 GPU 进行相似性搜索时出现的问题。第 4 节介绍了我们的主要贡献之一，即我们的 GPU *k* 选择方法，第 5 节介绍了整体算法的实现。最后，第 6 节为我们的方法提供了大量实验，将其与最新技术进行了比较，并展示了图像集合的具体使用案例。

2 问题陈述

我们关注的是向量集中的相似性搜索。给定一个查询向量 $x \in \mathbb{R}^d$ 和一个数据库向量 $y \in \mathbb{R}^d$ ，我们按 L_2 （欧几里得）距离搜索 x 的 h 个近邻：

$$L = \arg\min_{i=0, \dots, d} \|x - y_i\|_2 \quad (1)$$

L_2 距离最常用，因为它的线性代数特性很有吸引力，在学习多个嵌入时可以优化设计[26]。

通过 *h*-selection 方法收集最小距离。对于标量数组 $[a_i]_{i=0, \dots, h}$ ，*h*-selection 可以找到 h 个最小值元素 $[a_{s_i}]_{i=0, \dots, (h)}$ ， $a_{(s_i)_{(j)}} \leq a_{(s_i)_{(j+1)}}$ ，以及索引 $[s_i]_{i=0, \dots, (h)}$ 。
 $0 \leq s_{(j)} < l$ ，这些元素来自输入数组。 a_i 将是 32 位浮点数值； s_i 是 32 或 64 位整数。有时还需要其他比较器；对于余弦相似度，我们搜索最高值。等值 $a_{s_i} = a_{(s_i)_{(j)}}$ 之间的顺序没有指定。

2.1 带批处理的精确搜索

通常情况下，搜索以 n_q 次查询分批进行
矢量 $[x_j]_{j=0, \dots, n_q}$ ($x_j \in \mathbb{R}^d$) 并行运行，这样可以获得更多的
当在多个 CPU 线程上或
GPU。h 选择的批处理需要从 $n_q \times h$ 独立数组中选择 $n(q)h$ 个元素和索引，其中每个数组的长度 $l_i \geq h$ 可能不同。

精确解法是计算全部成对距离
矩阵 $D = [\|x_j - y_{(i)}\|_2^2]_{j=0, \dots, n_q, i=0, \dots, l} \in \mathbb{R}^{n_q \times l}$ 。在实践中，我们使用分解

$$\|x_{(j)} - y_{(i)}\|_2^2 = \|x_j\|_2^2 + \|y_{(i)}\|_2^2 - 2\langle x_j, y_{(i)} \rangle \quad (2)$$

前两个项是在矩阵 X 和 Y 的行 $[x_j]$ 和 $[y_{(i)}]$ 上一次性预先计算的。第

1. 为了避免基于 0 索引的混乱，我们使用 Python 数组符号 $0:l$ 来表示范围 $\{0, \dots, l-1\}$ 包括在内。

瓶颈在于评估 $x_j \cdot y_{(i)}$ ，相当于矩阵乘法 XY^T 。沿 D 的每一行为 $n_{(q)}$ 个查询选择 h 个近邻。

2.2 压缩域搜索

从现在起，我们专注于近似近邻搜索。我们特别考虑了 *IVFADC* 索引结构[32]。IVFADC 索引依赖于两级量化，并对数据库向量进行编码。数据库向量 y 的近似值为

$$y \approx q(y) = q_{(1)}(y) + q_{(2)}(y - q_{(1)}(y)), \quad (3)$$

其中 $q_1: \mathbb{R}^d \rightarrow C_1 \subset \mathbb{R}^d$ 和 $q_2: \mathbb{R}^d \rightarrow C_2 \subset \mathbb{R}^d$ 是量化函数，即从有限集中输出一个元素的函数。由于集合是有限的， $q(y)$ 被编码为 $q_{(1)}(y)$ 的索引和 $q_{(2)}(y - q_{(1)}(y))$ 的索引。第一级量化器为**粗量化器**，第二级量化器为**细量化器**，对第一级量化后的剩余向量进行编码。

非对称距离计算 (ADC) 搜索法返回近似结果：

$$L_{ADC} = h\text{-argmin}_{(i)=0:d} \|x - q(y_i)_{(2)}\| \quad (4)$$

对于 IVFADC 来说，搜索并不是详尽无遗的。计算距离的向量是根据一级量化器 q_1 预选的：

$$L_{IVF} = t\text{-argmin}_{c \in C_1} \|x - c\|_2 \quad (5)$$

多探针参数 t 是我们考虑的粗级中心点数量。量化器在重现值集合中进行精确距离的近邻搜索。然后，IVFADC 搜索计算出

$$L_{IVFADC} = h\text{-argmin}_{i=0:t \text{ s.t. } q_{(1)}(y_{(i)}) \in I_{IVF}} \|x - q(y_{(i)})_{(2)}\| \quad (6)$$

因此，IVFADC 依赖于与 ADC 的两步量化相同的距离估计，但只在向量子集上进行计算。

相应的数据结构，即**反转文件**，将向量 y_i 分成 $|C_1|$ 反转列表 $\mathbf{f}_1, \dots, \mathbf{f}_{|C_1|}$ 与同源的

基因的 $q_{(1)}(y_{ij})$ 。因此，最耗费内存的操作是计算 L_{IVFADC} ，相当于线性扫描 t 个反转列表。

量化器 q_1 和 q_2 具有不同的特性。量化器 q_1 需要有相对较少的重现值，这样倒置列表的数量才不会爆炸。

对于 q_2 ，我们可以负担更多的内存，以实现更广泛的计算。我们通常使用 $|C_2|$ 表示法。向量索引（4 或 8 字节整数）也存储在反转列表中，因此没有必要使用比它更短的代码；即 $\log_2 |C_2| > 4 \times 8$ 。

乘积量化器。我们使用**乘积量化器** (PQ) [32]来进行编码。 q_2 ，以有限的内存和计算成本提供大量的重现值。它将 y 解释为 b 个子向量 $y = [y^{(1)} \dots y^{(b)}]$ ，其中 b 是维数 d 的偶数除数。每个子向量用自己的量化器量化，得到 $(q^{(1)}(y^{(1)}), \dots, q^{(b)}(y^{(b)}))$ 。子量化器通常有 256 个重现值，可容纳在一个字节中。然后，乘积量化器的量化值

$q_{(2)}(y) = q^{(1)}(y^{(1)}) + 256 \times q^{(2)}(y^{(2)}) + \dots + 256^b \times q^{(b)}(y^{(b)})$ ，从存储角度看，这只是每个子量化器产生的字节的连接。因此，乘积 quan-

量化器生成的 b 字节编码具有 $|C_2| = 256^b$ 重现值。量化器的 h -means 字典很小，量化的计算成本很低。

3 图形处理器概述和 κ 选择

本节回顾了 Nvidia GPU 架构和编程模型的主要细节[40]。然后，我们将重点关注相似性搜索中与 GPU 兼容较少的部分，即 *h-selection*，并讨论相关文献和挑战。

3.1 架构

GPU 通道和翘曲。Nvidia GPU 是一种通用计算机，使用 32 宽的 *CUDA 线程向量* (warp) 执行指令流。战线中的单个线程称为**线程**，**线程 ID** 范围为 0 - 31。单个 warp 中的线程共享单个 warp 范围内的指令计数器。尽管共享指令计数器，但如果 warp 通道希望采用不同的执行路径，就会出现 *warp 分歧*，从而降低性能。每个通道在共享寄存器文件中最多有 255 个 32 位寄存器。与 CPU 类似，每个 warp 是一个独立的 CPU 硬件线程，拥有多达 255 个宽度为 32 的 SIMD 向量寄存器，而 warp 通道则是 SIMD 向量通道。

经线集合。可配置的 1 至 32 个经编器集合包括一个**区块**或**合作线程阵列**(CTA)。每个区块都有一个高速**共享内存**，大小可达 48 KiB。单个 CUDA 线程有一个与区块相关的 ID，称为**线程 ID**，可用于分区和分配工作。每个区块在 GPU 的一个内核上运行，该内核称为**流式多处理器**(SM)，由 ALU 等功能单元执行。GPU 通过在所有 SM 上对翘曲进行许多飞行操作来隐藏执行延迟。

每个单独的经线指令吞吐量较低，延迟较高，但所有 SM 的总算术吞吐量比典型 CPU 高 5-10 倍。

网格和内核。区块在**内核**中以**区块网格**的形式组织。每个块都有一个网格相对 ID。内核是主机 CPU 安排 GPU 执行的工作单元（带有参数的指令流）。区块运行完成后，可以调度新的区块。来自不同内核的块可以同时运行。内核之间的排序可通过**流**和**事件**等排序原语来控制。

资源和占用。执行的区块数量

同时取决于共享内存和寄存器

每个区块使用的资源。CUDA 线程的寄存器使用量在编译时确定，而共享内存的使用量可在运行时选择。这会影响到 GPU 的**占用率**；使用的寄存器或共享内存资源越多，执行并发性就越低。

内存类型。不同的区块和内核通过**全局内存**进行通信，**全局内存**的大小通常为 4- 32 GB，带宽比 CPU 主内存高 5- 10 $\times$ 。共享内存的速度类似于 CPU 的一级缓存。GPU 寄存器文件内存是带宽最高的内存。为了在 GPU 上保持大量的飞行指令，需要一个

还需要庞大的寄存器文件：最新的 Pascal P100 需要 14 MB，而 CPU 只有几十 KB。在 GPU 上，寄存器、共享内存和全局内存的总横截面带宽之比通常为 250 : 6.25 : 1，寄存器文件的速度为 10 - 100 TB/s[12]。

3.2 GPU 寄存器文件的使用

结构化寄存器数据。共享内存和寄存器内存的使用涉及效率权衡。它们降低了占用率，但通过在速度更快的内存中保留更大的工作集提高了整体性能。大量使用寄存器数据，以牺牲占用率或取代共享内存为代价，通常是有利可图的[54]。

由于 GPU 寄存器文件非常大，存储结构化数据（而不仅仅是临时操作数）非常有用。单个通道可以使用其（标量）寄存器解决本地任务，但并行性和存储能力有限。相反，翘曲中的车道可以使用**翘曲洗牌**交换寄存器数据，从而实现翘曲范围内的并行性和存储。我们提供了多种访问方式（移位、任意对任意）。其中，我们广泛使用了蝴蝶排列[39]。

巷跨寄存器阵列。翘曲洗牌经常用于操作**巷道寄存器阵列**。也就是说，给定元素 $[a_{(i)}]_{i=(0):(l)}$ ，每个连续值由相邻车道保存在寄存器中。数组存储在每个通道的 $l/32$ 个寄存器中，其中 l 是 32 的倍数。车道 j 存储 $\{a_j, a_{32+j}, \dots, a_{l-(32)+j}\}$ ，而寄存器 r 保存 $\{a_{(32)(r)}, a_{(32)(r)+1}, \dots, a_{32r+31}\}$ 。

要操作 $[a_{(i)}]_j$ ，必须在汇编时知道存储 a_i 的寄存器（即 $\lfloor i/32 \rfloor$ ）和 l ，而可以在运行时知道泳道（即 $i \bmod 32$ ）。因此，所有配置都必须在编译时通过广泛的 C++ 模板化来处理。

3.3 CPU 与 GPU 上的 k 选择

h 选择是先前 GPU 相似性搜索应用的一个性能限制因素（见第 6 节），因此值得讨论。常见的 CPU **h 选择**算法（通常适用于任意大的 l 和 h ）都可以转换到 GPU 上，包括 **radix 选择**和 **bucket 选择**[1]、**概率选择**[44]、**quickselect** [17] 和 **截断排序**[50]。它们的性能主要取决于对输入的多次处理。对于相似性搜索，输入距离通常是即时计算的，或者只存储在小块中，而不是全部。完整的距离数组可能太大，无法容纳在任何内存中，而且在处理开始时，其大小可能是未知的，这就使得多次通过算法变得不切实际。此外，根据元素值对全局内存进行分区的算法（如 Quickselect）会导致过多的内存事务，因为整个经线的数据访问模式并不统一。Radix 选择没有分区，但仍需要多次处理。

堆并行。在相似性搜索中，人们通常只关心少量（ $h < 1000$ ）结果。在这种情况下，通过最大堆（max-heap）进行选择是中央处理器上的一种典型方法，但由于要进行串行树更新，堆的数据并行性并不高，也无法使 SIMD 执行单元达到饱和。**ad-heap**[41]能更好地利用异构系统中的并行性，但仍试图在适当的执行单元之间划分串行和并行工作。尽管堆更新具有串行性质，但对于较小的 h ，CPU 可以在 L1 高速缓存中保持其所有状态，而不需要在 L1 高速缓存中进行更新。

而 L1 缓存的延迟和带宽仍然是一个限制因素。其他相似性搜索组件，如 PQ 代码处理，往往对 CPU 性能影响更大[3]。

GPU 堆。堆可以在 GPU 上实现[9]，但直接在 GPU 上实现的堆具有较高的翘曲发散性和不规则、依赖数据的内存移动，因为每个插入元素的路径取决于堆中存在的其他值。

GPU 并行优先级队列[31]允许多次并发更新，比串行堆更新有所改进，但每次插入都需要潜在数量的小排序，而且内存移动与数据相关。它们还需要与 CPU 主机进行大量协调。

其他更新颖的 GPU 算法可用于小型堆，即 *fgknn* 库中的选择算法[51]。这是一种复杂的算法，可能存在同步点过多、内核启动开销较大、使用较慢的内存、过度使用层次结构、分层和缓冲等问题。不过，我们从这一特殊算法中汲取了灵感，使用了并行合并，正如其**合并队列**结构所示。

4 GPU 上的快速 k 选择

对于任何 CPU 或 GPU 算法而言，按照**性能模型**[59]，内存或运算吞吐量都应该是限制因素。对于来自全局内存的输入，**h-selection** 的运行速度不能超过在内存带宽峰值时扫描一次输入所需的时间。我们的目标是尽可能接近这一极限。因此，我们对输入数据进行一次扫描。

我们希望在最快的内存（即寄存器文件）中保留中间状态。这样做的主要缺点是第 3.2 节中提到的寄存器阵列索引限制，从而限制了算法的可行性。

4.1 寄存器内排序

我们使用寄存器内排序原语作为构建模块。排序网络通常用于 SIMD 架构[16]，因为它们利用了矢量并行性。它们很容易在 GPU 上实现，我们使用巷式寄存器阵列构建排序网络。

我们使用的是**巴彻位子排序网络**[10]的一种变体，它是大小为 2^h 的数组上的一组并行合并。每次合并使用 $\log_{(2)}(l)$ 并行步骤，将 s 个长度为 t 的数组（ s 和 t 均为 2 的幂次）合并为 $s/2$ 个长度为 $2t$ 的数组。位排序递归地应用这种合并：要对长度为 l 的数组进行排序，需要将长度为 1 的 l 个数组合并为长度为 2 的 $l/2$ 个数组，再合并为长度为 4 的 $l/4$ 个数组，然后依次合并为长度为 l 的 1 个已排序数组，这样需要 $\lceil \log_{(2)}(l) \rceil$ 个并行合并步骤。

奇数大小的合并和排序网络。如果某些输入数据已经排序，我们可以修改网络以避免合并步骤。如果我们没有完整的 2 次幂数据集，我们可以通过高效的捷径来处理较小的数据量。

算法 1 是一个奇数大小的合并网络，它可以合并已经排序的左右两个数组，每个数组的长度都是任意的。比特网络合并**比**序列，而我们则从**单调**序列开始：单调排序的序列。通过反转第一个比较器阶段，位调合并就变成了单调合并。

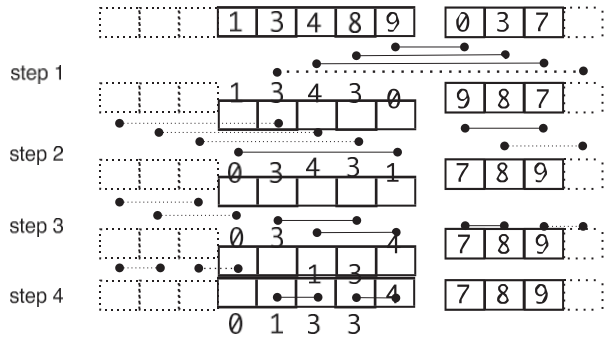


图 1. 奇数大小网络合并 5 号和 3 号数组。圆点表示并行比较/交换。虚线表示省略元素或比较。

算法 1. 奇数大小合并网络

```

函数 MERGE-ODD( $[L_{(l)}]_{l=(0):(l_f)}$ ,  $[R_{(l)}]_{l=(0):(l_f)}$ )
    parallel for  $i \leftarrow 0 : \min(l_L, l_R)$  do
         $\rho$  反相第一阶段；输入已排序
        COMPARE-SWAP( $l_{(l)}(i)$ ,  $r_{(l)}(i)$ )
    结束
    并行
         $\rho$  如果  $l_L = l_R$  和一个 2 的幂次，它们是等价的
        MERGE-ODD-CONTINUE( $[L_{(l)}]_{l=(0):(l_f)}$ , left)
        (MERGE-ODD-CONTINUE( $[R_{(l)}]_{l=(0):(l_f)}$ , right)
    end do
结束函数

函数 MERGE-ODD-CONTINUE( $[x_{(l)}]_{l=(0):(l)}$ ,  $p$ )
    if  $l > 1$  then
         $h \leftarrow 2^{(\log_2(l) - 1)}$   $\rho$  最大的 2 次幂  $< l$ 
        parallel for  $i \leftarrow 0 : l - h$  do
             $\rho$  使用翘曲洗牌实现
            COMPARE-SWAP( $x_i$ ,  $x_{i+h}$ )
        end for 并行
        do
            如果  $p = \text{left}$  then  $\rho$  左侧递归
                MERGE-ODD-CONTINUE( $[x_{(l)}]_{l=(0):(h)}$ , left)
            ODD-CONTINUE( $[x_{(l)}]_{l=(h):(l)}$ , right)
            否则  $\rho$  右侧递归
                MERGE-ODD-CONTINUE( $[x_{(l)}]_{l=(0):(h)}$ , left)
                CONTINUE( $[x_{(l)}]_{l=(h):(l)}$ , right)
            end if
        end do
    end if
结束函数

```

奇数大小算法是通过将数组填充到下一个最高的 2 次幂大小来推导的，填充的虚数元素从不交换（合并是单调的），并且已经正确定位；任何与虚数元素的比较都被省略。左数组在开始时被认为填充了哑元素；右数组在结束时填充了哑元素。将两个长度分别为 l_L 和 l_R 的排序数组合并为一个排序数组。和 l_R 的两个排序数组合并为一个长度为 $l_L + l_R$ 的排序数组，需要 $\log_{(2)}(\max(l_L, l_R)) + 1$ 并行步骤。图 1 显示了算法 1 对大小为 5 和 3 的数组的合并网络，并行步骤为 4。

COMPARE-SWAP 是在车道-跨度寄存器阵列上使用 warp shuffles 实现的。跨度为 32 倍的交换直接在车道内进行，因为车道本地保存两个元素。跨度为 16 倍或非 2 的幂的交换通过 warp shuffle 实现。实际上从 IEEE Xplore 下载。适用限制。

算法 2 将合并扩展为完全排序。假设没有输入数据中存在的结构，需要 $(1) \log_{(2)}(l) + \log_{(2)}(l) \log_{(2)}(l)$ 个并行步骤对长度为 l 的数据数组进行排序。

算法 2. 奇数大小排序网络

```

函数 SORT-ODD( $[x_{(l)}]_{l=(0):(l)}$ )
    if  $l > 1$  then parallel do
        SORT-ODD( $[x_{(l)}]_{l=0:(l/2)}$ )
        SORT-ODD( $[x_{(l)}]_{l=l/2:l}$ )
    end do
    MERGE-ODD( $[x_{(l)}]_{l=0:(l/2)}$ ,  $[x_{(l)}]_{l=l/2:l}$ )
    如果结束
结束函数

```

4.2 翘曲选择

我们的 *hselection* 实现 WARPSELECT 完全在寄存器中保持状态，只需要对输入进行一次传递。它使用 MERGE-ODD 和 SORT-ODD 作为原语。由于寄存器文件提供的存储空间远大于共享内存，因此它支持 $h \leq 1024$ 。每个 warp 专用于对 n 个数组 $[a_{(i)}]$ 中的一个进行 *h-selection*。如果 n 足够大，每个 $[a_i]$ 单个 warp 就能完全占用 GPU。如果事先知道 l ，则可以通过递归分解来处理每个 warp 的大小。

概述。我们的方法（算法 3 和图 2）对值进行操作，同时携带相关的索引（为简化描述，此处省略）。它从全局内存中选择最少的 h 个值，如果与提供值的另一个内核融合，则从中间值寄存器中选择最少的 h 个值。让 $[a_{(i)}]_{i=(0):(l)}$ 作为选择序列。

元素（图 2 左侧）以 32 为一组进行处理，即经线大小。小道 j 负责处理元素 $\{a_i, a_{32+i}, \dots\}$ 。因此，如果元素来自全局内存，则读取是连续的，并凝聚成最小数量的内存事务。每当其中一条通道满时（触发 WARP-BALLOT），所有通道都会调用合并例程，并刷新线程队列。

算法 3. j 车道的 WARPSELECT 伪代码

```

函数 WARPSELECT( $a$ )
    如果  $a < W_h$ ，则  $\rho$  每条车道  $j$  独立执行此操作
         $T_C^j \leftarrow a$ 
         $C_j \leftarrow C_j + 1$ 
    end if
    if WARP-BALLOT( $C_j = l$ ) then
         $\rho$  将线程队列重新解释为车道数组
         $[a_i]_{i=0:32l} \leftarrow \text{cast}([T_C^j]_{j=0:32l})$ 
         $\rho$  翘曲将线程队列排序在一起，更新  $W_i$ 
        SORT-ODD( $[a_{(l)}]_{l=(0:32l)}$ )
    end if

```

使用的数组长度是 32 的倍数，因为它们保存在车道-跨度组中。

MERGE-ODD($[W_{(i)}]_{i=(0):(h)}$, $[a_{(i)}]_{i=(0:32)(i)}$)
 $[T_i]_{i=0:T_f=0:32} \leftarrow +\infty$ $[C_j]_{j=}$ /
0:32 0
如果结束
结束函数

数据结构。翘曲共享一个由 h 个最小元素组成的巷道寄存器数组
, $[W_{(i)}]_{i=(0):(h)}$, 称为翘曲

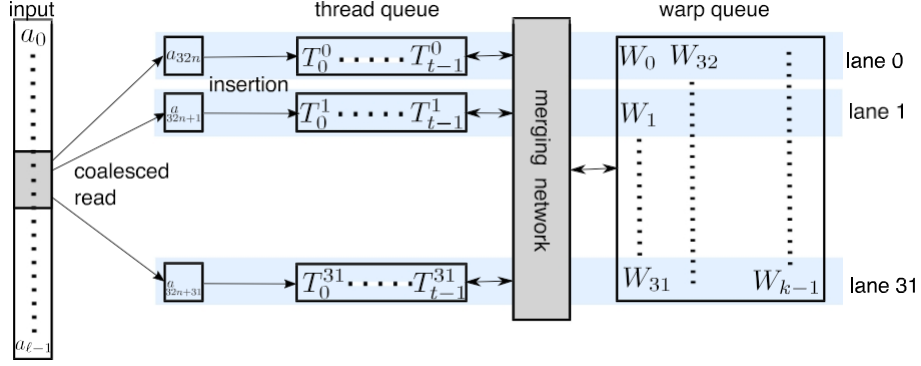


图 2. WARPSELECT 概述。左侧为输入值流，右侧为输出结果的翘曲队列。

队列。它的排序从小到大 ($W_i \leq W_{(i+1)}$)；如果请求的 h 不是 32 的倍数，我们就四舍五入。

每条车道 j 在寄存器中维护一小群 t 元素，称为线程队列 $[T]$ ，以及线程队列的计数。

当前有效的元素 $C_j (0 \leq C_j \leq t)$ 的数量。线程队列中索引为 $< C_j$ 的所有元素都保证是 $\leq W_{h(-)(1)}$ 。其他元素初始化为最大哨兵值，如 $+\infty$ 。

线程队列是新值进入的第一级过滤器；扫描期间只保留新的潜在 $\min-h$ 元素，直到我们在任何通道中收集到一组 t 元素，需要将其视为可能的真正 $\min-h$ 元素。 t 的选择相对于 h 而定，参见第 4.3 节。线程队列是第二层，它保存了所有 $\min-h$ 线程范围内的观测值。线程队列的初始化也是最大哨兵值。

更新维护的三个不变式是

- 所有 $T_i \leq W_{h(-)(1)}$ 对于 $i < C_j, T_i = +\infty$ 否则；
- 没有线程会在其线程队列中累积超过 t 个元素 $C_j \leq t$ ；
- 到目前为止，在真正的 $\min-h$ 中看到的所有 a_i 都包含在或某个车道的线程队列 $[T^j]$ 。

或

$$i = 0: C_j = 0: 32$$

在线程队列中。

车道 j 收到一个新的 a_{32i+j} ，我们将其与当前的 W_h 进行比较。如果 $a_{32i+j} > W_h$ ，则根据定义，新元素是 $W_{(h)}$ 。

不在 $\min-h$ 中，则剔除。否则，我们将其加入

线程队列，覆盖之前在线程队列中的哨兵值。如果任何车道已累积了 t 个值，那么我们就不能

处理任何新的 a_i 时，我们都无法确定它是否在目前看到的所有元素的 $\min-h$ 中，因为如果它是 $< W_h$ ，我们就没有地方保留这个元素。使用翘翘选票指令，我们可以确定是否有车道累积了 t 个值，在这种情况下，选票“获胜”。如果没有，我们可以继续处理新元素。

保持不变。现在，线程队列中的部分或全部元素都可能处于真正的最小 h 值。为了让 W_h 成为迄今为止看到的真正第 h 个最小元素，翘翘板使用 ODD-MERGE 将线程队列和翘翘板队列合并并排序。新的 warp 队列将是合并排序队列中的第 h 个最小元素，而线程队列则被重新初始化为最大哨兵值。这样，我们就可以在不违反不变式的情况下继续处理后续元素。

经队列已经排序，但线程队列尚未排序。线程队列一起排序，集合

支持奇数大小的合并是很重要的，因为巴彻的公式要求 $32t = h$ 是 2 的幂次。因此，如果 $h = 1024$ ， t 必须是 32。我们发现

最佳 t 值要小得多（见下文），这意味着我们正在合并两种不同的大小。

处理余数。如果因为 l 不是 32 的倍数而存在余数元素，那么这些元素将被视为覆盖余数的线程队列，之后我们将进入输出阶段。

输出。处理完所有元素后，对线程队列和经线队列进行最后的排序和合并，然后经线队列保存所有 a_i 的 $\min-h$ 值。

4.3 复杂度和参数选择

对于每组 32 个元素，WARPSELECT 会进行 1、2 或 3 次恒时操作，所有操作都在整个 warp 范围内的并行时间内进行：

- 1) 读取 32 个元素，与 W 比较，计算 C ，发生 N_1 次；
- 2) 插入线程队列，成本 C_2 ，发生 N_2 次；
- 3) 如果 E_j 使 $C_j = t$ ，则排序和合并队列的成本 N_3 。

$$C_3 = O(t \log(32t) + h \log(\max(h, 32t))), \text{ 发生 } N_3$$

次。

因此，总成本为 $N_1 C_1 + N_2 C_2 + N_3 C_3$ 。 $N_1 = l/32$ 。对于

N_2 ，因为我们将 $[T_i]$ 保留在寄存器中，这需要编译时索引。

编译时索引，因此我们使用对 t 的不滚动循环来查找适当的寄存器，以覆盖当前的 C_j 值，因此

$N_2 = O(t)$ 。对于 N_3 ，我们得出了独立抽取的随机数据的估计值。的估计值。

假设 h 选择的输入是一个序列 $\{a_1, a_2, \dots, a_l\}$ （基于 1 的索引），即一组不同元素的随机选择排列。元素按大小为 m 的 c 组顺序读取（翘翘板，因此 $m = 32$ ）。假设 l 是 m 的倍数，因此 $c = l/m$ 。回想一下， t 是线程队列的最大长度。我们将目前看到的 $\min-h$ 中 n 位置之前或 n 位置处的元素称为连续 $\min-h$ (n 处)。 a_n 在 n 处的连续 $\min-h$ 中的可能性为：

$$a_i(h) = \begin{cases} 1 & \text{if } (n) \leq h \\ h/n & \text{如果 } n > h \end{cases} \quad (7)$$

因为每个 $a_n, n > h$ 都有 h/n 的几率，因为所有排列的几率相同，而且

前 h 个元素都符合条件。

*线程队列插入计数。*在给定的线程中，如果输入的值在连续的线程队列中，则会触发插入。

+ $mc_0 + (c - c_0)$ 值，其中 c_0 是前一个赢得的 warp 选票。发生这种情况的概率为

$$a(mc_0 + (c - c_0), h + t) \approx \frac{h+t}{mc} \quad \text{对于 } c > h_0 \quad (8)$$

近似认为线程队列已看到所有 mc 值，而不仅仅是分配给其车道的 mc 值。因此，任何车道触发队列插入的概率都是：

$$1 - \frac{h}{mc} \pm \frac{t^{(m)}}{mc} \approx \frac{h+t}{c} \quad (9)$$

这里的近似值是一阶泰勒展开。

将 c 的概率相加，得到的预期插入次数为 $N_2 \approx (h+t) \log(c) = O(h \log(l/m))$ 。

计算全排序。 我们寻求 $N_{(3)} = p(l, h, t, m)$ ，WARPSELECT 所需的全排序的预期次数。

WARPSELECT 所需的全排序的预期次数。

单车道。 现在，我们假设 $m=1$ ，因此 $c=l$ 。让

$g(l, m, h)$ 是一个序列 $\{a_1, \dots, a_{(l)}\}$ 中，按顺序遇到的元素中正好有 m 个元素的概率。

扫描 $\text{next}(m=1)$ 是在连续的 $\text{min-}h$ 中。给定 m ，有 t 个地方可以

发生。它由一个递推关系给出：

$$\begin{aligned} g(l, m, h) &= \sum_{t=0}^{\min(h, l-m)} g(l-1, m-1, h-t) \cdot a(l, h-t) \\ &\quad + \sum_{t=0}^{\min(h, l-m)} g(l-1, m, h-t) \cdot (1-a(l, h-t)) \end{aligned} \quad \text{否则。} \quad (10)$$

最后一种情况是遇到以下情况的概率：有一个 $l-1$ 序列，在我们之前有 $m-1$ 个连续的 $\text{min-}h$ 元素，当前元素在连续的 $\text{min-}h$ 中，或者当前元素不在连续的 $\text{min-}h$ 中，在我们之前有 m 个元素。

然后，我们建立 $p(l, h, t, 1)$ 的递推关系。

我们首先注意到

$$d(l, b, h, t) := \sum_{m=b}^{\min((bt + \max(0, t-1)), l)} g(l, m, h), \quad (11)$$

为 b ，其中 $0 \leq b \leq l$ 是所有长度为 l 的序列中，通过赢得线程队列投票而强制获得 b 种数据的序列的分数，因为在连续的 $\text{min-}h$ 中必须有 bt 到 $(bt + \max(0, t-1))$ 个元素才能实现这些排序（因为 $\text{min-}h$ 元素会溢出线程队列）。因此， $p(l, h, t, 1)$ 是对所有可能的 b 的期望值：

$$p(l, h, t, 1) = \sum_{b=l}^{\lfloor l/t \rfloor} b \cdot d(l, b, h, t). \quad (12)$$

这个量可以通过动态编程计算出来。分析可知，对于 $t=1, h=1$ ， $p(l, 1, 1, 1)$ 是哈单数 $H_l = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{l}$ 收敛于

$\ln(l) + \gamma$ （欧拉-马斯切罗尼常数 γ ），当 $l \rightarrow \infty$ 时。

对于 $t=1, h>1, l>h$ ， $p(l, h, 1, 1) = h + h(H_{(l-h)}(h))$ 或 $O(h \log(l))$ ，因为前 h 个元素在连续最小

对于 $t>1, h>1, l>h$ ，注意到有一定数量的 $D, h \leq D \leq l$ 对每个可能的 $\{a_{(1)}, \dots$ 的连续 $\text{min-}h$ 确定 $D_0, a_{(D)}\}$ 。根据定义，每种情况的获胜选票数为 D/t ，因为线程队列必须填充 t 次。因此， $p(l, h, t, 1) = O(h \log(l)/t)$ 。

多车道。 $m>1$ 的情况因以下事实而变得复杂

需要考虑的是联合概率（如果多于一个 m 工作者触发了给定组的排序，则只进行一次排序）。不过，这种可能性是有界的。

设 $p^{(i)}(l, h, t, m)$ 是预期中选的选票，假设没有

m 名工人在赢得选票时相互干扰（即，如果有 $b \leq m$ 名工人在单步中不间断地赢得一张选票，则我们赢得 b 张选票），但由于共享的“.....

从联合序列中每次排序后的 $\text{min-}h$ 集。假设 $h \geq m$ 。那么我们有

$$\begin{aligned} p^{(i)}(l, h, 1, m) &\leq \sum_{i=1}^m \frac{h}{m \cdot i \cdot l} + \frac{h}{m(h/m + t)} \\ &\leq m p(l/m, h, 1, 1) = O(mh \log(l/m)), \end{aligned} \quad (13)$$

在这种情况下， m 名工人看到连续的 $\text{min-}h$ 元素的可能性

最小 h 元素的上限是第一个工人的上限。

在每一步。与之前一样，中选票数按 t 缩放，因此 $p^{(i)}(l, h, t, m) = O(mh \log(l/m)/t)$

t ，所以 $p^{(i)}(l, h, t, m) = O(mh \log(l/m)/t)$ 。相互干扰只能减少选票数，因此我们得到相同的

$p(l, h, t, m)$ 的上限。假设 m 固定为经

大小，我们有 $N_3 = p(l, h, t, 32) = O(h \log(l)/t)$ 。

在 $N_{(2)} C_2$ 和 $N_{(3)} C_{(3)}$ 中权衡成本。在给定 h 和 l 的情况下，我们通过对各种 h - N 数据的实验，对 t 做出了切实可行的选择。对于 $h \leq 32$ ，我们

使用 $t=2, h \leq 128$ 使用 $t=3, h \leq 256$ 使用 $t=4, h \leq 1024$

使用 $t=8$ ，都与 l 无关。

5 在 GPU 上实现索引

本节将解释第 2 节中相似性搜索方法在 GPU 上的高效实现，其中特别关注针对最大数据库的 IVFADC。它

是最初建立在产品量化基础上的索引方法之一[32]。

量化方法之一[32]。距离计算的细节以及与 h 选择的衔接是理解以下内容的关键

为什么这种方法能胜过最近的 GPU 近似近邻策略[58]。

5.1 精确搜索

我们再来简单谈谈穷举搜索法，也就是通常所说的精确蛮力搜索法。

对于小数据集的精确近邻搜索，这种方法本身就很有趣。它也是文献中许多索引的组成部分；我们将其用于 IVFADC 粗量化器 q_1 。

如第 2 节所述，距离计算可归结为矩阵乘法。我们使用 cuBLAS 库中的优化 GEMM 例程来计算 $-2\langle x_i, y_i \rangle$ 项。

项与 L_2 距离的关系，从而得出一个部分不平衡的结果。

距离矩阵 D' 。为了完成距离计算，我们使用融合 h 选择核，将 $\|y_i\|^2$ 项加入每个

h ，其余的期望值为 $\frac{h}{m} + \frac{1}{m} + \frac{(h)}{m}$ 。

部分距离结果，并立即将该值提交给寄存器中的 $h\text{-selection}$ 。 $\|x_j\|^2$ 项 无需在 $h\text{selection}$ 之前考虑。
可与其他 GPU 计算融合的 $h\text{selection}$ 只允许进行 2 次处理
授权许可使用仅限于South China Normal University.于 2025 年 7 月 13 日 08:51:09 UTC 从 IEEE Xplore 下载。适用限制。

(相比之下, 其他实现通常需要 3 个或更多的矩阵 D' (GEMM 写入, h -select 读取)。

由于矩阵 D' 无法在 GPU 内存中满足实际问题规模的要求, 因此问题被分块覆盖在一批查询中, $t_q \leq n_{(q)}$ 个查询在一个分块中运行。

5.2 IVFADC 索引

PQ 查找表。 IVFADC 需要计算一个向量到一组 PQ 重现值的距离。通过对数据库向量 y 建立方程 (6), 我们可以得到

$$\|x - q_{(1)}(y)\|_2^2 = \|x - q_{(1)}\|_2^2 + \|q_{(2)}(y - q_{(1)}(y))\|_2^2 \quad (14)$$

如果我们将 $q_{(1)}$ 后留下的残差向量分解为

$$y - q_{(1)}(y) = [y^{xcl} \dots y^{xeb}], \text{ 并且} \quad (15)$$

$$x - q_{(1)}(y) = [x^{xcl} \dots x^{xeb}]_1 \quad (16)$$

则距离改写为

$$\|x - q(y)\|_2^2 = \|x^{xcl}\|_2^2 + \dots + \|x^{xeb} - q^{(xeb)}\|_2^2 \quad (17)$$

每个量化器 $q^1, \dots, q^b$ 有 2^{56} 个再现值。 q^b 有 256 个重现值, 因此当已知 x 和 $q_{(1)}(y)$ 时, 所有距离都可以预先确定并存储在表 $T_1, \dots, T_b$ 中 [32]。 T 表中, 每个表的大小为 $256 \times b$ [32]。

计算总和 (17) 包括 b 次查找和加法运算。比较计算 n 个距离的成本:

- 显式计算: $n \times d$ multiply-adds;
- 使用查找表: $256 \times d$ 乘法加法和 $n \times b$ 查找加法。

这是 PQ 效率的关键。在我们的 GPU 实现中, b 是 4 到 64 的任意倍数。代码在列表中以每个向量 b 字节为一组顺序存储。

IVFADC 查找表。 当扫描反转列表元素 $\mathbf{f}_i$ 时, 可以使用查找表方法, 因为查询 x 是已知的, 而且根据定义 $q_1(y)$ 是常数。更多在此基础上, 表格 $T_{(1)} \dots T_b$ 的计算进一步优化 [6]。公式 (14) 中 $\|x - q(y)\|_2^2$ 的表达式分解为:

$$\|q_{(2)}(\dots)\|_2^2 = \|q_{(1)}(y) - q_{(2)}(\dots)\|_2^2 + \|x - q_{(1)}(y)\|_2^2 + 2 \langle x - q_{(1)}(y), q_{(2)}(\dots) \rangle \quad (18)$$

我们的目标是尽量减少内循环计算。我们可以提前进行并存储在查找表中的计算如下:

- 术语 1 与查询无关。它可以通过量化器预先计算出来, 并存储在大小为 $|C_1| \times 256 \times b$ 的表 Y 中;
- 第 2 项是与 q_1 重现值的距离。因此, 它是一级量化器 q_1 的副产品;
- 项 3 的计算与反转列表无关。它的计算代价是 $d \times 256$ 个乘加。

该分解用于生成在扫描反向列表时使用的查找表 $T_{(1)} \dots T_{(b)}$ 。对于单次查询, 从头开始计算 $t \times b$ 表需要 $t \times d \times 256$ 次乘法加法, 而这种分解需要 $256 \times d$ 次乘法加法和 $t \times b \times 256$ 次加法。在 GPU 上

Y 的内存用量可能会非常大, 因此我们只在不需要考虑内存的情况下才启用分解。

5.3 GPU 实现

算法 4 总结了在 CPU 上的执行过程。反转列表存储为两个不同的数组, 分别代表 PQ 代码和相关的 ID。只有当 h 选择确定 h 最近成员时, ID 才会被解析。这种查找是在一个大数组中读取一些稀疏的内存, 因此对于 GPU 来说, ID 可以选择存储在以下位置 CPU, 以降低性能成本。

算法 4. IVFPQ 批量搜索程序

```

函数 IVFPQ-SEARCH( $[X_1, \dots, X_{(n)}]_{(q)}$ ,  $\mathbf{f}_1, \dots, \mathbf{f}_{|C_1|}$ )
  for  $i \leftarrow 0 : n_q$  do
     $L_{IVF}^i \leftarrow \text{t-argmin}_{(c) \in C_{(i)}} \|x - q\|_2$ 
  end for
  for  $i \leftarrow 0 : n_q$  do
     $L \leftarrow []$ 
    计算项 3 (见第 5.2 节)
    对于  $L$  中的  $\sqrt{d}$  do
      计算距离表  $T_1, \dots, T_b$ 
      for  $j \in \mathbf{f}_i$  do
         $d \leftarrow \|x - q(y_j)\|_2^2$ 
        将  $(d, L, j)$  追加到  $L$ 
      end for
       $R_i \leftarrow k$ -从  $L$  中选择最小距离  $d$ 
    end for
    返回  $R$ 
  结束函数
  
```

列表扫描。 内核扫描每个查询的 t 个最接近的反向列表, 并使用查找表 T_i 计算每个向量对的距离。 T_i 存储在共享内存中: 一个查询集最多需要 $n_q \times t \times \max_i |\mathbf{f}_i| \times b$ 次查询 (实际中需要数万亿次访问), 而且是随机访问。这就限制了当前 GPU 架构下的 b 最多为 48 (32 位浮点) 或 96 (16 位浮点)。如果我们不

的分解, T_i 的计算方法是

在扫描前由一个单独的内核进行处理。

多路内核。 每个 $n_q \times t$ 对查询和反向列表可以独立处理。一个极端是

每个块都有一个专用块, 结果是最多有 $n_q \times t \times \max_i |\mathbf{f}_i|$ 部分结果被写回全局内存, 然后再经过 h 次选择得到 $n_q \times h$ 个最终结果。这将产生很高的并行性, 但可能会超出 GPU 可用的全局内存。与精确搜索一样, 我们选择瓦片大小 $t_q \leq n_q$ 来减少内存消耗, 在多流情况下, 其复杂度边界为 $O(2t_q t \max_{(i)} |\mathbf{f}_{(i)}|)$ 。

端到端性能。 对于我们的 IVFADC + PQ 实现, 表 1 显示了相对于 Maxwell Titan X GPU 的屋顶线模型峰值的全局内存带宽和算法利用率。表 1 是在 YFCC100M 索引 (第 6.4 节) 上对 4096 个向量的查询进行的评估, 其中 $t = 32$, $h = 100$, 没有对 T_i 进行分解。端到端工作负载是相当异构的, 既有较高的算术需求 (IVFADC q_1), 也有较高的内存带宽需求 (列表扫描)。

表 1
GPU IVFADC+ PQ 端到端性能

时间百分比（ 共 1.3 秒）	内核	算术（峰值 百分比）	内存（ 峰值百分 比）	限制因素
19.4%	IVFADC q_1	95%	44%	算术
5.9%	h-select q_1	40%	64%	gmem b/w
23.4%	T_i 距离	60%	66%	gmem b/w
34.5%	列表扫描	26%	87%	黑白计算机
14.7%	h 选择列表	55%	52%	算术
2.1%	ID 查找	23%	13%	实例延迟
加权平均		52.4%	65.7%	gmem b/w

时间为 1.3 秒，但 h-select q_1 内核和 h-select list 内核都利用了内核融合的优势。否则，就需要在全局内存中进行多次传递，从而使工作负荷由 *h-selection* 主导。

5.4 多 GPU 并行性

现代服务器可支持多个 GPU。我们利用这一功能来提高速度和内存。

复制。如果一个索引适合单个 GPU 的内存，那么它可以在 R 个不同的 GPU 上复制。要查询 n_q 个向量，每个副本只需处理查询量的 n_q/R 部分。复制的速度接近线性，但对于较小的 n_q 可能会降低效率。

分片。如果一个索引无法容纳在单个 GPU 的内存中，可以将索引分片到 S 个不同的 GPU 中。在添加 l 个向量时，每个分片接收 l/S 个向量；在查询时，每个分片处理完整的查询集 n_q ，并将部分结果（仍需要额外的 h 轮选择）合并到单个 GPU 或 CPU 内存中。对于给定的索引大小 l ，分片会提高速度（分片对 l/S 的查询量为 n_q ，而复制对 l 的查询量为 n_q/R ），但由于固定开销和后续 *hselection* 的成本，速度通常低于纯复制。

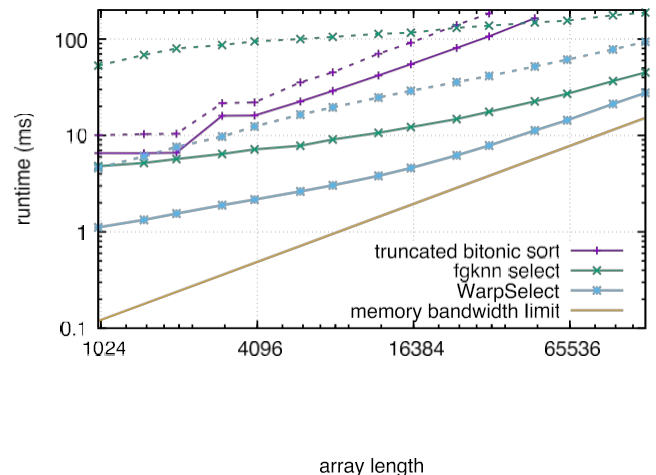
复制和分片可以同时使用（ S 个分片，每个分片有 R 个副本，共 $S \times R$ 个 GPU）。分片或复制都非常简单，同样的原理也可用于在多台机器上分配索引。

6 实验和应用

本节将对我们的 GPU *hselection* 和近邻方法与现有库进行比较。除非另有说明，否则实验都是在 $2 \times 2.8\text{GHz}$ Intel Xeon E5-2680v2 和 4 个使用 CUDA 8.0 的 Maxwell Titan X GPU 上进行的。

6.1 k 选择性能

我们将其与其他两种 GPU 小型 *hselection* 实现进行比较：从 Tang 等人[51] 的 *fgknn* 库中提取的基于行的合并队列（Merge Queue with Buffered Search）和分层分区（Hierarchical Partition），以及从 Sismanis 等人[50] 的截断比托尼排序（*TBiS*）中提取的基于行的合并队列（Merge Queue with Buffered Search）和分层分区（Hierarchical Partition）。两者都是从各自的精确搜索库中提取的。之所以选择这些实现，是因为它们不需要对输入数据进行多次排序。任何需要对输入数据进行多次遍历的实现都会在很大程度上受到全局内存带宽的限制



，而且在我们的使用率超过 50% 的情况下，这些实现都无法胜出。
fgknn 和 *TBiS*

图 3.不同 h 选择方法的运行时间与数组长度 l 的函数关系。同时处理的数组数量为 $n_q=10000$ 。全线 $h=100$ ，虚线 $h=1000$ 。

与我们的实现方法不同，我们的方法需要额外的临时全局内存用于中间计算。

我们在单个 Titan X 上评估了行大矩阵 $n_q \times l$ 中每行随机 32 位浮点数值的 $h=100$ 和 1000 。批量大小 n_q 固定为 10000 ，数组长度 l 从 1000 到 128000 不等。问题的输入和输出都保留在 GPU 内存中，输出大小为 $n_q \times h$ ，并有相应的索引。因此，输入问题的大小从 40 MB ($l=1000$) 到 5.12 GB ($l=128k$) 不等。TBiS 需要较大的辅助存储空间，在我们的测试中，其容量被限制在 $l \leq 48k$ 。

图 3 显示了我们与 TBiS 和 fgknn 的相对性能。图 3 显示了我们与 TBiS 和 fgknn 的相对性能，其中还包括 Titan X 内存带宽限制所能达到的峰值性能。WARPSELECT 相对于 fgknn 的性能随着 h 的增大而增加；甚至在 $h=1000$ 时，TBiS 的性能也开始超过 fgknn。我们尤其关注最大 $l=128000$ 。WARPSELECT

在 $h=100$ 时比 $\times$ 快 1.62 ，在 $h=1000$ 时比 $\times$ 快 2.01 。WARPSELECT 的运行速度在 $h=100$ 时为峰值的 55% ，而在 $h=1000$ 时仅为峰值的 16% 。这是因为当 h 较大时，线程队列和合并/排序网络会增加额外开销。

与 fgknn 的区别 WARPSELECT 受到了 fgknn 的影响，但也有一些改进：所有状态都保存在寄存器中（没有共享内存），没有星际同步，使用多内核启动或缓冲，没有“分层分割”，奇数大小的网络提供了更高效的合并和排序。这些改进使得 h -selection 可以直接融合到其他 GPU 内核中，这对于精确相似性搜索（第 5.1 节）和 IVFADC 列表遍历（第 5.2 节）具有显著的性能优势。

6.2 k-均值聚类

在分配阶段， $h=1$ 的精确搜索可用于 h -means 聚类方法，将 n_q 个训练向量分配到 $|C_1|$ 中心点。尽管它没有使用 IVFADC，而且 $h=1$ 的选择是微不足道的（是并行的 min-reduction，而不是 WARPSELECT），但 h -means 是用于训练量化器 q_1 的聚类的良好基准。

我们在 MNIST8m 图像上应用了该算法。这 810 万张图像是 28×28 像素的灰度数字，线性化为 784 -d 的向量。在表 2 中，我们比较了这

表 2
MNIST8m h-Means 性能

方法	# GPU	# 中心点	
		256	4096
BIDMach [13]	1	320 s	735 s
我们的	1	140 s	316 s
我们的	4	84 s	100 s

两种算法都运行了 20 次迭代。虽然两者都是基于 cuBLAS 实现的，但我们的实现速度要快 2× 以上。我们的实现从 h 选择融合到 L_2 距离计算中获得了一些好处。在通过复制执行多 GPU 时，对于足够大的问题，速度提升接近线性（4096 个中心点的 4GPU 速度为 3.16 倍）。需要注意的是，这个基准有点不切实际，因为当要求的中心点如此少时，通常会随机对数据集进行子采样。

大规模。我们还与 Avrithis 等人的近似方法[4]进行了比较。该方法将 $10^8 128$ -d 向量聚类为 85K 个中心点。他们的聚类方法运行时间为 46 分钟，但至少需要 56 分钟的预处理来对向量进行编码。我们的方法无需任何预处理，在 4 个 GPU 上执行精确 h -means 只需 52 分钟。

6.3 精确近邻搜索

我们考虑一个用于评估近邻搜索的经典数据集：SIFT1M [32]。其特征大小为 $l = 10^6, d = 128, n_q = 10^4$ 。计算部分距离矩阵 $D^{(i)}$ 的成本为 $n_q \times l \times d = 1.28$ Tflop，在当前的 GPU 上运行时间不到一秒。图 4 显示了距离计算的峰值与方程 (2) 中 $-2 < x_j, y_i >$ 项的 GEMM 平铺成本的对比，以及在大小为 $n_q \times l$ 的距离矩阵上可能达到的峰值 h 选择性能，其中还考虑了在峰值内存带宽下读取平铺结果矩阵 $D^{(i)}$ 的情况。

除了第 5 节中的方法外，我们还包括了第 6.1 节中评估 h 选择性能的两个 GPU 库的时间。我们得出以下几点结论

- 对于 h -selection，使用 `thrust::sort_by_key` 对整个结果数组进行排序的天真算法要比较方法慢 10× 以上；
- 在所有方法中， L_2 距离和 h 选择成本都占主导地位，但我们的方法除外，假设使用 GEMM，我们在 GEMM 上对部分距离矩阵 D' 进行平铺，其性能达到峰值的 85%，接近最优。cuBLAS GEMM 本身对于较小的缩减规模（ $d = 128$ ）效率较低；

- 我们的融合 L_2 和 h 选择内核非常重要。同样的算法，如果不进行融合（需要额外通过 D' ）至少为 25 较慢。

2.BIDMach 数字 来自 <https://github.com/BIDDData/BIDMach/wiki/Benchmarks#KMeans>

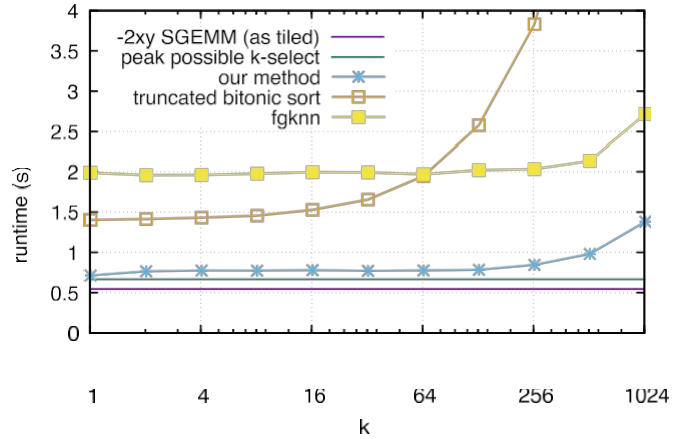


图 4.在 1 个 Titan X GPU 上对不同 h 的 SIFT1M 数据集进行精确搜索 h -NN 所需的时间。

在使用近似方法计算距离的情况下，高效的 h 选择更为重要，因为相对于距离计算， h 选择的相对成本会增加。

6.4 十亿级近似搜索

关于大型数据集（ 10^{10} ）上近似近邻搜索的研究很少。在此，我们使用该领域的标准数据集和评估协议，对索引搜索进行了一些比较。表 3 提供了这些数据集的统计数据。我们对 SIFT1B 和 DEEP1B 最感兴趣，据我们所知，这两个数据集是用于相似性搜索任务的最大公开数据集。

SIFT1M。为了完整起见，我们首先比较了我们在 SIFT1M 上的 GPU 搜索速度和 Wieschollek 等人的实现[58]。他们在 Titan X 上每次查询 0.02 毫秒内获得的最近邻检索率为 1（真正最近邻在前 1 个结果中的查询比例）： $R@1 = 0.51, R@100 = 0.86$ ；在相同的时间预算内，我们的实现获得了 $R@1 = 0.80$ 和 $R@100 = 0.95$ 。

SIFT1B。我们再次与 Wieschollek 等人在 SIFT1B 数据集[33]上进行比较，该数据集包含 10 亿个 SIFT 图像特征， $n_q = 10^4$ 。我们以相同的内存使用量来比较相似精度下的搜索性能（更精确的方法可能需要更长的搜索时间或内存使用量）。在单 GPU 上，每个向量 $m = 8$ 字节，每个查询向量的 $R@10 = 0.376$ ，耗时 17.7 毫秒，而他们报告的每个查询向量的 $R@10 = 0.35$ ，耗时 150 毫秒。因此，我们的实现更准确，速度快 8.5×。

DEEP1B。我们还在 DEEP1B 数据集[8]上进行了实验，该数据集包含 10 亿个 CNN 表示的图像， $n_q = 10^4$ 。介绍该数据集的论文报告了 CPU 结果（1 个线程）： $R@1 = 0.45$ ，每个向量的搜索时间为 20 毫秒。我们

表 3
评估数据集的属性

数据集	# 数据集 向量	# 查询 向量	# 训练 向量	dims	数据大小
SIFT1M	1000000	10000	100000	128	128 MiB
SIFT1B	1000000000	10000	100000000	128	128 GiB
深度1B	1000000000	10000	350000000	96	384 GB
YFCC100M	95074575	n/a	n/a	128	48.6 GiB

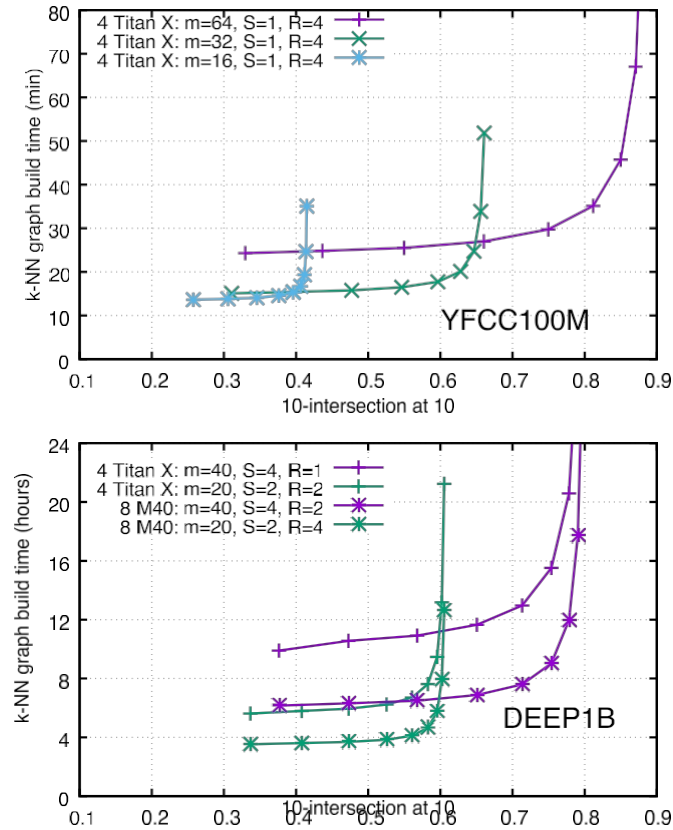


图 5. 针对 YFCC100M 和 DEEP1B 数据集的 10-NN 强制图构建的速度/精度权衡。

使用 $m=20$ 的 PQ 编码, $d=80$ 通过 OPQ [23], 以及 $|C_i|=2$,¹⁸ 其使用的数据集存储量与原始论文 (20 GB) 相当。这需要多个 GPU, 因为对于单个 GPU 的全局存储器来说, 数据集太大了, 所以我们考虑使用 4 个 GPU, $S=2$, $R=2$ 。我们获得了 $R@1=0.4517$, 每个向量耗时 0.0133 毫秒。虽然硬件平台不同, 但这表明在 GPU 上进行搜索在单台机器上实现的速度方面是一个游戏规则的改变。

YFCC100M. 该数据集 [52] 包含 9920 万张图片和 80 万个视频。我们可以下载其中的 9500 万张图片。我们将 CNN 描述子计算为 ResNet [30] 的最后一层之前的一层, 并通过 PCA 缩减为 $d=128$ 。由于我们仅将其用于 h-NN 图实验, 因此不区分训练集和查询集。

6.5 k-NN 图

我们相似性搜索方法的一个应用实例是通过蛮力 (根据整个索引查询所有向量) 构建数据集的 h 近邻图。

实验设置. 我们在 YFCC100M 和 DEEP1B 数据集上评估了速度、精度和内存之间的权衡:

- **速度:** 通过搜索数据集中所有向量的近邻, 从头开始建立 IVFADC 索引并构建整个 h -NN 图 ($h=10$) 所需的时间。因此, 这是一个端到端的测试, 包括索引和搜索时间;

- **质量:** 我们对 10,000 幅图像进行采样, 计算出准确的近邻图像。我们测量返回的 10 个近邻中, 在地面实况 10 个近邻范围内的近邻比例。

对于 YFCC100M, 我们使用粗量化器 (2^{16} 个中心点), 并为每个矢量考虑 $m=16$ 、32 和 64 字节 PQ 编码。对于 DEEP1B, 我们通过 OPQ 将向量预处理为 $d=120$, 使用 $|C_i|=2^{18}$, 并考虑 $m=20$ 、40。对于给定的编码, 我们在 1 到 256 之间改变 t , 以在效率和质量之间取得权衡, 如图 5 所示。在 Deep1B 上进行的 k-NN 图实验中, 我们没有使用训练集和查询向量, 而是从主数据集中进行了采样。我们使用两种相当常见的多 GPU 工作站配置进行了数据密集型应用实验: 4 个 Maxwell 级 Titan X GPU 或 8 个 M40 GPU。

讨论. 对于 YFCC100M, 我们使用 $S=1$, $R=4$ 。精确度超过 0.8 只需 35 分钟。对于 DEEP1B, 质量较低的图可以在 6 小时内构建, 质量较高的图大约需要半天时间。我们还尝试使用更多 GPU, 将副本集增加一倍, 使用 8 个 Maxwell M40 (M40 的性能大致相当于 Titan X)。性能得到了亚线性提升 ($\sim 1.6\times$ for $m=20$, $\sim 1.7\times$, $m=40$)。

相比之下, 我们所知的最大 h -NN 图构建使用的数据集包含 3,650 万个 384-d 向量, 使用 NN-Descent [18], 128 台 CPU 服务器集群需要计算 108.7 个小时 [56]。需要注意的是, NN-Descent 也可以建立或完善 h-NN 图。

在我们所考虑的数据集中, DEEP1B 的搜索时间较短, 但它对图存储的内存开销较大, DEEP1B 的图存储空间已达 80 GB。此外, 它还需要对所有向量进行随机访问 (DEEP1B 为 384 GB)。

我们发现的最大型 GPU h -NN 图构建方法是使用 GEMM 精确搜索, 对包含 2 千万个 15,000-d 向量的数据集进行粗暴构建, 32 个 Tesla C2050 GPU 集群花费了 10 天时间 [17]。假设计算量与距离矩阵的 GEMM 成本成比例, 那么 DEEP1B 的这种方法在他们的集群上需要 200 天的计算时间, 这是不现实的。

6.6 使用 k-NN 图

在为图像数据集构建了 h-NN 图之后, 只要存在单个连接组件 (就是这种情况), 我们就能在图中找到任意两幅图像之间的路径。例如, 我们可以通过传播从起始图像到目标图像的邻居, 搜索两幅花卉图像之间的最短路径。用 2 和 D 表示源图像和目的图像, d_{ij} 表示节点之间的距离, 我们搜索路径 $P=\{p_1, \dots, p_n\}$, 其中 $p_1=2$, $p_n=D$, 这样:

$$\min_{P} \max_{i=1, \dots, n} d_{(p_i) (p_{i+1})} \quad (19)$$

也就是说, 我们希望平稳过渡。图 6 显示了 YFCC100M³ 的结果示例。这是在 h-NN 图中传播 20 秒后得到的结果, $h=15$ 个邻居。由于数据集中有许多花朵图像, 因此过渡非常平滑。

3.3. DEEP1B 没有从向量到图像的映射。



图 6.来自 YFCC100M 的 9500 万张图像的 h-NN 图中的路径。图中给出了第一张图像和最后一张图像，算法计算出它们之间最平滑的路径。

7 结论

GPU 的算术吞吐量和内存带宽已达到每秒太拉福尔和数百千兆字节。

然而，实现接近这些性能水平的算法既复杂又违背直觉。在本文中，我们介绍了在 GPU 上实现接近最佳性能的相似性搜索方法的算法结构。

这项工作使以前需要复杂近似算法的应用成为可能。例如，本文介绍的方法使精确的 h -means 聚类或使用简单的暴力方法计算 h-NN 图成为可能，所需的时间比 CPU（或 CPU 集群）近似计算所需的时间更短。

这项工作的局限性是 GPU 架构所固有的。GPU 以吞吐量为导向、非延迟优化的执行模型对于暴力计算或内存阵列的线性扫描（如 IVFADC）非常有效。其他近似的 h-NN 方法，如最近的基于图的方法 HNSW [42] 和 NSG [22]，都依赖于指针追逐和稀疏内存访问，很可能无法高效地映射到 GPU 硬件上。就像我们在 GPU SIMD 架构上实现高效 h 选择的工作一样，我们应该探索从看似串行的算法中提取并行性的巧妙方法，看看 HNSW 和 NSG 是否值得在 GPU 上使用。此外，GPU 的内存仍然比典型 CPU 服务器的 RAM 少一个数量级。在这种内存受限的环境下，采用更多量化和压缩技术来扩展内存/速度权衡工具箱也将被证明是非常有用的。

由于机器学习算法的普及，GPU 硬件目前在科学工作站中非常常见。我们相信，我们的工作进一步证明了 GPU 在大数据应用中的重要性。

参考文献

- [1] T. Alabi, J. D. Blanchard, B. Gordon, and R. Steinbach, "Fast k-selection algorithms for graphics processing units," *ACM J. Exp. 算法*, 第 17 卷, 第 4.2.4.1-4.2.4.29 页, 2012 年 10 月。
- [2] A. Andoni and P. Indyk, "Near-optimal hashing algorithms for near neighbor problem in high dimensions," in *Proc. Symp. Foundations Comput. 科学*, 2006 年, 第 459-468 页。
- [3] F. Andrieu, A.-M. Kermarrec, and N. L. Scouarnec, "Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan," in *Proc. Int. Conf. 超大型数据库*, 2015 年, 第 288-299 页。
- [4] Y. Avrithis, Y. Kalantidis, E. Agnostonopoulos, and I. Z. Emiris, "Web-scale image clustering revisited," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2015 年, 第 1502-1510 页。
- [5] A. Babenko and V. Lempitsky, "The inverted multi-index," in *Proc. Comput. Pattern Recognit. Pattern Recognit.*, pp. 3069-3076.
- [6] A. Babenko and V. S. Lempitsky, "Improving bilayer product quantization for billion-scale approximate nearest neighbors in high dimensions," *CoRR*, abs/1404.1831, Apr.
- [7] A. Babenko and V. Lempitsky, "Aggregating local deep features for image retrieval," in *Proc. Comput. Pattern Recognit. Pattern Recognit.*, 2015 年, 第 1269-1277 页。
- [8] A. Babenko and V. Lempitsky, "Efficient indexing of billion-scale datasets of deep descriptors," in *Proc. Comput. Pattern Recognit. Pattern Recognit.*, 2016 年 6 月, 第 2055-2063 页。
- [9] R. Barrientos, J. Gomez, C. Tenllado, M. Prieto, and M. Marin, "knn query processing in metric spaces using GPUs," in *Proc. Int. Eur. Parallel Distrib. Parallel Distrib. Comput.*, Sep. 2011, pp.
- [10] K. E. Batcher, "Sorting networks and their applications," in *Proc. Spring Joint Comput. Conf.*, 1968, pp.
- [11] P. P. Boncz, W. Lehner, and T. Neumann, "特刊: VLDB J., vol. 25, no. 5, pp.
- [12] J. Canny, D. L. W. Hall, and D. Klein, "A multi-teraflop constituency parser using GPUs," in *Proc. Empirical Methods Natural Language Process.*, 2013 年, 第 1898-1907 页。
- [13] J. Canny and H. Zhao, "Bidmach: BigLearn Workshop, 2013.[在线]. 可用网址: <https://nips.cc/Conferences/2013/Schedule?showEvent=3698>
- [14] M. Caron, P. Bojanowski, A. Joulin, and M. Douze, "Deep clustering for unsupervised learning of visual features," *CoRR*, abs/1807.05520, Jul. 2018.
- [15] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proc. ACM Symp. Theory Comput.*, pp. 380-388.
- [16] J. Chhugani, A. D. Nguyen, V. W. Lee, W. Macy, M. Hagog, Y.-K. Chen, A. Baransi, S. Kumar, and P. Dubey, "Efficient implementation of sorting on multi-core simd CPU architecture," *Proc.*, 1, no. 2, pp.
- [17] A. Dashti, "Efficient computation of k-nearest neighbor graphs for large high-dimensional data sets on GPU clusters," Master's thesis, Engineering, Univ. Wisconsin Milwaukee, Milwaukee, WI, Aug. 2013.
- [18] W. Dong, M. Charikar, and K. Li, "Efficient k-nearest neighbor graph construction for generic similarity measures," in *Proc. World Wide Web*, Mar. 2011 年 3 月, 第 577-586 页。
- [19] M. Douze, H. Jegou, and J. Johnson, "An evaluation of large-scale methods for image instance and class discovery," in *Proc. ACM Multimedia*, 2017 年, 第 1-9 页。
- [20] M. Douze, H. Jegou, and F. Perronnin, "Polysems codes," in *Proc. Eur. Conf. Comput. Vis.*, 2016 年 10 月, 第 785-801 页。
- [21] M. Douze, A. Szlam, B. Hariharan, and H. Jegou, "Low-shot learning with large-scale diffusion," *CoRR*, abs/1706.02332, Jun. 2017.
- [22] C. Fu, C. Xiang, C. Wang, and D. Cai, "Fast approximate nearest neighbor search with the navigating spreading-out graph," *Proc. VLDB Endowment*, vol. 12, no. 5, pp.
- [23] T. Ge, K. He, Q. Ke, and J. Sun, "Optimized product quantization," *IEEE Trans. Pattern Anal. Mach. Intell.*, 4, pp.
- [24] Y. Gong and S. Lazebnik, "迭代量化: A procrustean approach to learning binary codes," in *Proc. Comput. Vis. Pattern Recognit.*, 2011 年 6 月, 第 817-824 页。
- [25] Y. Gong, L. Wang, R. Guo, and S. Lazebnik, "Multi-scale orderless pooling of deep convolutional activation features," in *Proc. Eur. Conf. Comput. 视觉*, 2014 年, 第 392-407 页。

- [26] A.Gordo, J. Almazan, J. Revaud, and D. Larlus, "Deep image retrieval: Learning global representations for image search," in *Proc.Eur.Conf.Comput.Vis.*, 2016 年, 第 241-257 页。
- [27] A.Gordo, F. Perronnin, Y. Gong, and S. Lazebnik, "Asymmetric distances for binary embeddings," *IEEE Trans.Pattern Anal. 机器. Intell.*, 第 36 卷, 第 1 期, 第 33-47 页, 2014 年 1 月。
- [28] S.Han, H. Mao and W. J. Dally, "深度压缩: 用剪枝、训练量化和胡夫曼编码压缩深度神经网络", *CoRR*, abs/1510.00149, 2015 年 10 月。
- [29] K.He, F. Wen, and J. Sun, "K-means hashing: An affinity-preserving quantization method for learning binary compact codes," in *Proc.Comput.Vis.Pattern Recognit.*, 2013 年 6 月, 第 2938-2945 页。
- [30] K.He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc.Comput.Vis.Pattern Recognit.*, 2016 年 6 月, 第 770-778 页。
- [31] X.He, D. Agarwal, and S. K. Prasad, "Design and Implementation of a parallel priority queue on many-core architectures," in *Proc.Conf.High Perform.Comput.*, 2012, pp.
- [32] H.J' egou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Trans.Pattern Anal. 机器.* 33, no. 1, pp.
- [33] H.J' egou, R. Tavenard, M. Douze, and L. Amsaleg, "Searching in one billion vectors: Re-rank with source coding," in *Proc.Conf.声学、语音信号处理*, 2011 年 5 月, 第 861-864 页。
- [34] A.Joulin, E. Grave, P. Bojanowski, and T. Mikolov, "Bag of tricks for efficient text classification," arXiv:1607.01759, 2016.
- [35] Y.Kalantidis and Y. Avrithis, "Locally optimized product quantization for approximate nearest neighbor search," in *Proc. Comput. Vis. Pattern Recognit.*, 2014 年 6 月, 第 2329-2336 页。
- [36] A.Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proc.Advances Neural Inf.Process.Syst.*, 2012, pp.
- [37] B.Kulis and T. Darrell, "Learning to hash with binary reconstructive embeddings," in *Proc.Advances Neural Inf.Process.Syst.*, pp.1042-1050.
- [38] G. Lample, A. Conneau, M. Ranzato, L. Denoyer, and H. J' egou, "Word translation without parallel data," *CoRR*, abs/1710.04087, Oct. 2017.[Online].Available: <https://iclr.cc/Conferences/2018/Schedule?showEvent=336>
- [39] F.T. Leighton, *Introduction to Parallel Algorithms and Architectures: Array, Trees, Hypercubes*, San Francisco, CA, USA: Morgan Kaufmann, 1992.
- [40] E.E.Lindholm, J.Nickolls, S.Oberman and J.Montrym, "NVIDIA Tesla: 统一的图形和计算架构", 《IEEE Micro》, 第 28 卷, 第 2 期, 第 39-55 页, 2008 年 3 月。
- [41] W.Liu and B. Vinter, "Ad-head: 不对称多核处理器的高效堆数据结构", *Proc.Workshop Gen.使用GPU*, 2014 年, 第 54:54-54:63 页。
- [42] Y.A. Malkov and D. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *CoRR*, abs/1603.09320, Mar. 2016.
- [43] T.Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc.Advances Neural Inf.Process.Syst.*, pp.3111-3119.
- [44] L.Monroe, J. Wendelberger, and S. Michalak, "Randomized selection on the GPU," in *Proc.ACM Symp.High Perform.Graph.*, pp.89-98.
- [45] M.Norouzi and D. Fleet, "Cartesian k-means," in *Proc.Comput.Pattern Recognit.Pattern Recognit.*, 2013 年 6 月, 第 3017-3024 页。
- [46] J.Pan and D. Manocha, "Fast GPU-based locality sensitive hashing for k-nearest neighbor computation," in *Proc.ACM Int.Advances Geographic Inf.Advances Geographic Inf.Syst.*, 2011, pp.
- [47] L.Paulev' e, H. J' egou, and L. Amsaleg, "Locality sensitive hashing: 散列函数类型和查询机制的比较", 《模式识别》(Pattern Recognit. *Pattern Recognit.Lett.*, 第 31 卷, 第 11 期, 第 1348-1358 页, 2010 年 8 月。
- [48] O.Shamir, "Fundamental limits of online and distributed algorithms for statistical learning and estimation," in *Proc.Advances Neural Inf.Process.Syst.*, 2014, pp.
- [49] A.Sharif Razavian, H. Azizpour, J. Sullivan, and S. Carlsson, "CNN features off-the-shelf: An astounding baseline for recognition," in *Proc.Comput.Vis.Pattern Recognit.Work-shops*, 2014, pp.
- [50] N.Sismanis, N. Pitsianis, and X. Sun, "Parallel search of k-nearest neighbors with synchronous operations," in *Proc.Extreme Comput.Conf.*, 2012, pp.
- [51] X.Tang, Z. Huang, D. M. Eysers, S. Mills, and M. Guo, "Efficient selection algorithm for fast K-NN search on GPUs," in *Proc.Parallel Distrib.Process.Symp.*, 2015, pp.
- [52] B.Thomee, D. A. Shamma, G. Friedland, B. Elizalde, K. Ni, D.Poland, D. Borth, and L.-J. Li, "YFCC100M: 多媒体研究的新数据", 《Commun.ACM》, 第 59 卷, 第 2 期, 第 64-73 页, 2016 年 1 月。
- [53] G. Toliás, R. Sircé, and H. J' egou, "Particular object retrieval with integral max-pooling of CNN activations," *CoRR*, abs/1511.05879, Nov. 2015.[Online].Available: <https://iclr.cc/archive/www/doku.php%3Fid=iclr2016:accepted-main.html>
- [54] V.Volkov and J. W. Demmel, "Benchmarking GPUs to tune dense linear algebra," in *Proc.ACM/IEEE Conf.超级计算*, 2008 年, pp.31:1-31:11.
- [55] A.Wakatani and A. Murakami, "GPGPU implementation of nearest neighbor search with product quantization," in *Proc.Symp.Parallel Distrib.Process.应用*, 2014 年, 第 248-253 页。
- [56] T.Warashina, K. Aoyama, H. Sawada, and T. Hattori, "Efficient k-nearest neighbor graph construction using mapreduce for large-scale data sets," *Efficient k-Nearest Neighbor Graph Construction Using MapReduce Large-Scale Data Sets Trans.*, vol. 97-D, no, pp.3142-3154, 2014.
- [57] R.Weber, H.-J. Schek, and S. Blott, "A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces," in *Proc. Int.Conf.Very Large DataBases*, 1998, pp.
- [58] P.Wieschollek, O. Wang, A. Sorkine-Hornung, and H. P. A. Lensch, "Efficient large-scale approximate nearest neighbor search on the GPU," in *Proc.Comput.Pattern Recognit.Pattern Recognit.*, 2016 年 6 月, pp.2027-2035.
- [59] S.Williams, A. Waterman, and D. Patterson, "Roofline: An insightful performance model for multicore architectures," *Commun.ACM*, vol. 52, no.4, 第 65-76 页, 2009 年 4 月。



Jeff Johnson 于 1998 年获得普林斯顿大学计算机科学学士学位。他自 2014 年起在 Facebook 人工智能研究中心 (FAIR) 担任研究工程师, 自 2013 年起在 Facebook 工作。2001-2012 年间, 他致力于为视频游戏和互动媒体研究对性能敏感的实时物理模拟算法和低延迟分布式计算技术。在加入 FAIR 之前, 他在 Facebook 开发了用于网络规模应用的新型分布式数据库系统。他撰写了许多内部 Facebook 使用的 GPU 系统, 以及 PyTorch 深度学习框架中的外部 GPU 系统。他目前从事机器学习的硬件、软件和算法协同设计研究。



Matthijs Douze 于 2004 年获得法国高等工科大学计算机科学硕士学位和图卢兹大学计算机视觉博士学位。自 2015 年以来, 他一直是 Facebook AI Research (FAIR) 的研究科学家。2005-2015 年, 他是格勒诺布尔高等研究院 LEAR 项目团队的工程师。他的主要研究课题是计算机视觉和相似性搜索的大规模算法。



Hervé Jégou 于 2005 年获得雷恩大学博士学位, 他的论文题目是抗错压缩和联合源 channel

在参加了 École Normale Supérieure de Cachan。自 2015 年以来, 他一直是 Facebook AI Research (FAIR) 的研究科学家。此后, 他主要从事计算机视觉和模式识别方面的工作。他于 2006 年加入 INRIA, 成为一名长期研究员, 参与或领导了多个大型图像和视频集合项目。2018 年, 他获得了欧洲计算机视觉学会 (ECCV) 科恩-德林克时间测试奖 (Koen-derink Test of Time Prize)。他是电气和电子工程师学会 (IEEE) 的资深会员。

